You are a Senior Full-Stack Engineer, AI Architect, UI/UX Designer,
and Hackathon Project Specialist.

Build a complete, polished, functional, hackathon-ready application
called PLANB LAB.

Do not merely create a UI mockup. Implement working features,
test the core workflows, and prepare the application for deployment.

==========

1. PROJECT IDENTITY

==========

Project Name: PLANB LAB
Tagline: Break the plan before reality does.
Hackathon Theme: Decision Intelligence

Core Concept:
PLANB LAB is an AI-assisted decision intelligence platform that
helps people discover potential failures in their plans, understand
dependencies between failures, evaluate alternative solutions,
and test whether proposed contingency plans improve outcomes.

The application must be understandable to nontechnical users.

The experience should follow four steps:

UNDERSTAND → DISCOVER → IMPROVE → RETEST

Primary demonstration:

A college event registration workflow that depends on internet connectivity, OTP verification, QR scanning, and staff availability.

Important:

Do not claim that the idea is completely unprecedented.

Focus on delivering a distinctive, useful implementation.

==========

2. CRITICAL DEVELOPMENT CONSTRAINTS

==========

The project must be completed within a four-hour hackathon.

Priorities:

1. Working functionality.
2. Excellent UI/UX.
3. Correct and explainable simulation results.
4. Secure API integration.
5. Compliance with submission requirements.
6. Successful deployment.

Avoid unnecessary complexity.

Do not repeatedly regenerate working files.

Inspect the existing project before modifying it.

Reuse existing dependencies whenever possible.

Do not install packages unless necessary.
Do not spend time creating features that do not improve the
core demonstration.

Implement features incrementally and verify them.

If a feature is too complex, implement a smaller working version
rather than leaving a broken feature.

Do not stop after creating the frontend.
Complete the backend, database integration, and AI integration
to the extent possible within the time limit.

==========
3. REQUIRED TECHNOLOGY STACK
==========

Frontend:
- React
- Vite
- React Router
- Tailwind CSS
- Lucide React icons if already available or easy to install
- Recharts for charts if needed
- React Flow for the dependency graph if practical

Backend:
- Node.js

- Express.js
- JWT authentication
- bcrypt password hashing
- Zod validation

Database:
- Supabase PostgreSQL

AI:
- Google Gemini API

Deployment:
- Frontend: Vercel
- Backend: Render
- Database: Supabase

Version Control:
- Git
- Public GitHub repository

Inspect the existing environment first.
If a dependency is already installed, reuse it.
Prefer simple, maintainable implementations.

Never expose the Gemini API key, database service-role key,
JWT secret, or other private credentials in frontend code.

Store private keys only in backend environment variables.

Create .env.example with placeholder values.

Ensure .env and other secret files are excluded from Git.

==========

4. PREMIUM UI/UX DESIGN

==========

Create a polished, modern decision intelligence dashboard.

Visual direction:

- Dark navy background.
- Indigo and violet accents.
- Green for recovered or successful states.
- Amber for warnings.
- Red for failed states.
- Subtle gradients.
- Rounded cards.
- Soft borders and shadows.
- Clear typography.
- Smooth, purposeful transitions.
- Attractive but readable charts.

The application must feel like a coherent commercial product,

not a collection of unrelated generated components.

Create a responsive layout for desktop and mobile.

Desktop layout:

- Compact left navigation sidebar.
- Main workspace in the center.
- Contextual information panel on the right.
- Collapsible AI Copilot panel.
- Clear top navigation.

Sidebar:

- Dashboard
- My Workflows
- Risk Analysis
- Failure Map
- Simulations
- AI Copilot
- Chat History
- Settings

Use consistent spacing, reusable components, loading indicators,

empty states, error messages, and success notifications.

Avoid excessive animations that interfere with usability.

Do not sacrifice functionality for appearance.

==========

5. LANDING PAGE

==========

Create an attractive landing page with:

PLANB LAB logo and branding.

Headline:

"Every plan has a Plan B."

Subtitle:

"Discover hidden risks. Test your decisions. Prepare for failure."

Primary CTA:

"Test My Plan"

Secondary CTA:

"Explore Demo"

Provide sample workflow cards:

- College Event
- Small Business
- Emergency Planning

Allow users to enter a short description of their plan.

For example:

"I'm organizing a college fest for 500 students. Registration

requires internet access, OTP verification, and QR scanning."

Users should be able to start with the sample workflow immediately.

Do not force users to configure complicated settings before they can experience the application.

==========

6. DASHBOARD

==========

Build a functional dashboard containing:

A. Overall Plan Health

Show a clear status:

- Looks Resilient
- Needs Attention
- High Risk

Explain that this status is based on the currently configured risk assumptions and tested scenarios.

B. Summary Cards

- Components analyzed.
- Failure scenarios tested.
- Scenarios handled successfully.
- Unresolved failure scenarios.

C. Highest-Risk Components

Display the components with the highest configured risk.

D. Recent Investigations

Show previously saved workflows and their latest test results.

E. Quick Actions

- Create Workflow
- Run Stress Test
- View Failure Map
- Compare Decisions

All buttons must have meaningful functionality.

Do not display hardcoded dashboard metrics as if they were live simulation results.

Seed data may be used for the initial example, but clearly associate results with the sample workflow.

==========

7. WORKFLOW BUILDER

==========

Create an editable workflow builder.

Default workflow:

Online Registration

↓

OTP Verification

↓

QR Scanning

↓

Identity Verification

↓

Event Entry

Represent each component as a visual node.

Each node should support:

- Component name.
- Description.
- Dependencies.
- Configured failure probability.
- Conditional failure relationships.
- Current simulation state.
- Available fallback procedures.

Allow users to:

- Select a component.
- Inspect its details.
- Edit its configuration.
- Enable or disable fallback procedures.
- Save changes.
- Run a simulation.

Users must be able to understand the workflow without

understanding programming or graph theory.

For the four-hour MVP, support the supplied workflow template and a practical, limited set of editable fields.

Natural-language workflow generation is optional.
If implemented, use Gemini through the backend.

==========

8. FAILURE PROBABILITY ANALYSIS

==========

Implement transparent risk calculations.

Every component can have a configurable failure probability between 0 and 100 percent.

Provide clear labels:
- Configured Estimate
- Historical Estimate, if supported by real data.

Never invent historical data.

Never present arbitrary AI-generated percentages as reliable real-world failure predictions.

For the initial MVP, use configurable assumptions.

Example:

Internet connection failure probability: 20%.

Display this using:

- A clear percentage.
- A horizontal risk bar or circular indicator.
- A plain-language explanation.
- Potential consequences.
- Related components.
- Suggested interventions.

Allow users to edit the assumed probability.

Whenever an assumption changes, recalculate dependent results.

Clearly distinguish:

1. Probability of an individual component failing.
2. Conditional probability of another component failing
   given the first failure.
3. Probability of both events occurring.
4. Fraction of tested scenarios that failed.

Do not treat these four measurements as interchangeable.

For two events A and B:

$$P(A \text{ and } B) = P(A) \times P(B \mid A)$$

Use this formula only when the conditional probability is

defined and the inputs are appropriate.

If the events are independent, the joint probability may be calculated using the independence assumption.

Do not assume independence when a dependency exists.

For complex scenarios with unclear probabilities, report that the probability is unknown or assumption-dependent.

==========

9. FAILURE CONNECTION MAP

==========

Create an interactive dependency graph.

This is one of the application's main differentiators.

Show connections between components and explain how one failure could affect downstream components.

Example:

Internet Connection

├── OTP Verification

├── Online Records

└── Online QR Validation

↓

Event Entry

These dependencies must be configurable.

Do not assume that QR scanning always requires internet access.

Offline verification may be possible depending on the workflow.

Graph requirements:

- Selectable nodes.
- Labeled connections.
- Clear status colors.
- Highlighted affected components.
- Component details on selection.
- Explanations of dependency relationships.
- Failure propagation based on configured rules.

Use a graph visualization library if it can be integrated quickly.

Otherwise, build a simpler connected-node visualization.

Avoid building a complex graph editor if it threatens completion.

==========

10. FAILURE INJECTION ENGINE

==========

Implement a deterministic simulation engine.

Support these example failures:

1. Internet outage.

2. OTP service unavailable.

3. QR scanner unavailable.

4. Trained staff member unavailable.

Allow individual failures and combinations of failures.

The application must systematically generate non-empty combinations of the selected failure conditions.

For four failure conditions, there are 15 possible non-empty combinations.

Provide controls to:

- Select failure conditions.
- Run individual scenarios.
- Run combined scenarios.
- Limit the number of combinations tested.
- Inspect the result of each scenario.

Each scenario must produce:

- Initial failure conditions.
- Affected components.
- Failure propagation path.
- Successful steps.
- Degraded steps.
- Blocked steps.

- Final process outcome.

- Explanation of the result.

Use explicit workflow rules.

Do not use random results.

Do not pretend to execute real-world systems.

This is a simulation of a configured workflow.

Make the assumptions visible and editable.

==========

11. CONTINGENCY PLANNER

==========

Provide fallback options for the college registration workflow.

Examples:

A. Offline registration records.

B. Alternative identity verification.

C. Backup QR verification procedure.

D. Backup trained staff member.

Allow users to enable or disable these options.

Each fallback must have explicit activation conditions.

Example:

If internet access is unavailable:

- Activate offline registration records.
- Use the configured alternative verification method.
- Check whether trained staff are available.
- Allow the workflow to continue only if all required
  fallback conditions are satisfied.

Do not simply mark a scenario successful because a fallback has been enabled.

Evaluate the actual configured conditions.

Show:

- What the fallback changes.
- Which scenarios it can address.
- Which dependencies remain.
- Which failures it cannot resolve.

==========

12. APPLY FIX & RETEST

==========

This is the most important interactive feature.

Create a prominent button:

"APPLY FIX & RETEST"

Required behavior:

1. Preserve baseline configuration and results.
2. Let the user enable or modify contingency options.
3. Run the same scenarios against the revised configuration.
4. Compare baseline and revised results.
5. Identify recovered scenarios.
6. Identify scenarios that still fail.
7. Explain which changes caused the differences.

Display:

Before vs After

- Successful scenarios.
- Failed scenarios.
- Recovered scenarios.
- Remaining failure paths.
- Changed component states.

Calculate scenario success percentages from actual test results.

Example:
If 9 out of 10 scenarios succeed, display 90% of tested
scenarios handled successfully.

Do not describe this as a 90% probability of real-world success.

Keep the original results available for comparison.

Provide a way to reset the workflow to its baseline configuration.

==========

13. AI COPILOT

==========

Implement a contextual AI assistant called PLANB Copilot.

The Copilot should help users understand the workflow,

simulation results, dependencies, and potential improvements.

Example questions:

- Why is this component high risk?
- What happens if the internet fails?
- Which components depend on this service?
- Which scenario remains unresolved?
- What should we improve first?
- Explain the results in simple English.
- Compare two contingency options.

Use Gemini through the Express backend.

Send only the relevant workflow configuration and simulation
results required to answer the user's question.

The assistant must distinguish:
- Calculated results.
- User-configured assumptions.
- AI-generated recommendations.
- Unknown or unverified information.

Never allow the model to invent simulation outcomes.

For questions requiring a new simulation:
1. Request or perform the relevant simulation.
2. Provide its actual results to the AI.
3. Ask the AI to explain those results.

Keep prompts concise to minimize API usage.

Do not call the AI API for ordinary deterministic operations.

Handle missing API keys gracefully.
Provide a clear configuration message and retain the core
simulation functionality when AI access is unavailable.

Do not place API keys in frontend environment variables
that are exposed to the browser.

==========

14. CHAT HISTORY

==========

Create a chat interface with conversation history.

Users should be able to:

- Start a new conversation.
- View previous conversations.
- Reopen a conversation.
- See previous messages.
- Delete a conversation.
- Continue discussing a saved workflow.

Associate conversations with workflows when appropriate.

For the initial MVP, implement simple persistence first.

Prefer database-backed persistence when time permits.

Do not store private API keys or sensitive credentials
inside conversation history.

If authentication is implemented, enforce user ownership
of conversations on the backend.

==========

15. BACKEND AND AUTHENTICATION

==========

Implement a Node.js and Express backend.

Required capabilities:

- Registration.
- Login.
- JWT authentication.
- bcrypt password hashing.
- Request validation using Zod.
- Workflow endpoints.
- Simulation endpoints if needed.
- Chat endpoints.
- Conversation history endpoints.
- Appropriate error handling.

Validate all user-controlled inputs.

Use secure password hashing.

Never store plaintext passwords.

Never trust a user-supplied user ID to authorize access.

Derive the authenticated user identity from a verified token.

Use short-lived access tokens where practical.

Do not put secrets in source code.

Configure CORS using an environment variable containing

the allowed frontend origin.

Do not use permissive production settings unnecessarily.

If the time limit prevents complete authentication features, prioritize a secure, clearly scoped implementation rather than adding decorative or nonfunctional login screens.

==========

16. DATABASE

==========

Use Supabase PostgreSQL.

Design a minimal schema for:

A. Users, if authentication is implemented.
B. Workflows.
C. Workflow components and dependencies.
D. Saved simulation configurations and results.
E. Chat conversations and messages.

Store probabilities, assumptions, and results explicitly where appropriate.

Associate private records with their owner.

Use parameterized database operations or the official

Supabase client safely.

Do not expose service-role credentials in the frontend.

Document all required environment variables.

If database credentials are unavailable during development,
support a controlled local demonstration mode where possible.
Do not pretend that local demonstration data has been persisted
to the production database.

==========

17. API AND SECURITY REQUIREMENTS

==========

Create .env.example with placeholders for:

PORT

GEMINI_API_KEY

JWT_SECRET

SUPABASE_URL

SUPABASE_ANON_KEY

SUPABASE_SERVICE_ROLE_KEY, only if required by the backend

FRONTEND_URL

Use only the variables actually required by the implementation.

Ensure:

- .env is included in .gitignore.
- API keys never appear in client bundles.
- Authentication is verified on protected routes.
- Input validation is enforced.
- Errors do not expose secrets.
- Database permissions are appropriate.
- Public repository files contain no credentials.

Do not fabricate credentials.

If credentials are missing, clearly document the configuration needed to enable that integration.

==========

18. DEPLOYMENT AND SUBMISSION

==========

Prepare the project for deployment.

Frontend:
- Vercel-compatible configuration.
- Correct API base URL configuration.
- React Router fallback configuration if needed.

Backend:
- Render-compatible start script.
- Correct port handling.
- Health-check endpoint.

- Production CORS configuration.

Database:

- SQL schema or migration instructions.
- Required setup instructions.

GitHub:

- Clear project README.
- Setup instructions.
- Feature list.
- Architecture overview.
- Environment variable documentation.
- API endpoint documentation.
- Deployment instructions.
- Known limitations.
- Example workflow and simulation instructions.

Never commit .env files or private keys.

Do not claim that the application is deployed unless it
has actually been deployed and tested.

Do not claim that the AI works unless its integration
has been configured and verified.

==========

19. ERROR HANDLING AND TESTING

==========

Test the following:

1. The default workflow loads correctly.
2. Individual failure scenarios produce deterministic results.
3. Combined failures are handled correctly.
4. Dependencies propagate failures correctly.
5. Fallback rules change outcomes only when their conditions are satisfied.
6. Before-and-after comparisons use the same test scenarios.
7. Probabilities are calculated consistently.
8. Unknown probabilities are not fabricated.
9. Invalid inputs are rejected.
10. Missing AI credentials do not break the core simulator.
11. Chat history behaves correctly.
12. Authentication and authorization protect private data.
13. The production build succeeds.
14. API failures display useful error messages.

Fix critical errors before adding optional features.

==========

20. FOUR-HOUR PRIORITY ORDER

==========

Build in this order:

PHASE 1:

Inspect the environment and create the basic application.

PHASE 2:

Implement the workflow and deterministic simulation engine.

PHASE 3:

Implement failure combinations, dependency visualization,

and risk calculations.

PHASE 4:

Implement contingency options and Apply Fix & Retest.

PHASE 5:

Polish the dashboard and make the application intuitive.

PHASE 6:

Integrate backend, database, authentication, and Gemini.

PHASE 7:

Test, prepare documentation, and deploy.

Do not wait until the end to discover whether the project builds.

Verify the application after each major phase.

If the schedule becomes tight:

- Preserve the simulation engine.
- Preserve the failure map.

- Preserve the before-and-after comparison.
- Simplify optional visual effects.
- Avoid unnecessary dependencies.
- Avoid repeated AI calls.
- Do not add untested features near the deadline.

The core application must remain useful without Gemini.

==========

21. FINAL QUALITY STANDARD

==========

The final application must be:

- Visually impressive.
- Easy to understand.
- Responsive.
- Functional.
- Explainable.
- Honest about uncertainty.
- Efficient with API usage.
- Secure in its handling of credentials.
- Suitable for a live hackathon demonstration.

Every major button must work.

Every displayed metric must come from actual application

state, configured assumptions, or clearly labeled sample data.

Every simulation result must be reproducible under the same configuration.

The application must explain why a scenario succeeded or failed.

The AI must explain decisions rather than replace the simulation.

The user must be able to compare a baseline with an improved plan.

Review the existing PLANB LAB project against every requirement in the original hackathon specification.

Do not regenerate working code or duplicate existing features.

Ensure the project includes:

1. PROBLEM_STATEMENT.md
2. SOLUTION_DESCRIPTION.md
3. A complete README.md with setup and deployment instructions
4. A public GitHub repository preparation checklist
5. A working frontend and backend
6. Secure Gemini API integration through backend environment variables
7. JWT authentication, bcrypt password hashing, and request validation
8. Database integration
9. Vercel frontend and Render backend deployment configuration
10. A 3–5 minute demo video script and recording checklist.

Identify missing requirements and implement the highest-priority

ones without breaking existing functionality.

Do not claim deployment, testing, or video recording is complete unless it has actually been performed.

==========

22. START NOW

==========

First inspect the current project directory and existing code.

Identify:

- Existing framework and dependencies.
- Missing dependencies.
- Available environment configuration.
- Current implementation status.

Then create a concise implementation checklist and immediately start implementing the highest-priority working features.

Do not ask unnecessary questions.
Make sensible assumptions and document them.

Do not stop after planning.
Do not return only code snippets or design suggestions.

Build the actual application, run it, test it, and fix errors.

At the end, report:

1. Features implemented.

2. Features tested.

3. Commands to run the project.

4. Environment variables still required.

5. Deployment steps still pending.

6. Known limitations.

Remember:

We have four hours.

Build a reliable, impressive, working product—not an oversized,

unfinished collection of features.